A PROJECT REPORT

ORDEAL

Online Examination and Proctoring Platform
with Browser-Based Monitoring

Submitted by

Abhijeet Singh | Roll No. 2501920100016

Project repository: github.com/abhijeet0207hub/Ordeal

Repository revision reviewed: 86d1e38 (06 November 2026)

Report date: 12 November 2026

Degree, department, institution, supervisor, and academic year were not supplied; no details have been invented.

Declaration

This report describes the Ordeal software project and is based on inspection of the public repository, checked-in implementation, build metadata, and the sources cited in the references. It is prepared for review by the named project author. Before signing or submitting this report, the author should independently confirm its accuracy, authorship statements, originality requirements, and compliance with the applicable institution's rules.

Project author: Abhijeet Singh

Roll number: 2501920100016

Signature: _____

Date: _____ Place: _____

Academic program and institution details were not provided.

Certificate

This page is reserved for the institution’s authorized supervisor or department. No supervisor, department, or institution was identified in the information supplied for this report; therefore, this document does not certify institutional approval, originality, or completion on behalf of any authority.

Project title: Ordeal — Online Examination and Proctoring Platform with Browser-Based Monitoring

Student: Abhijeet Singh Roll No.: 2501920100016

Supervisor name and designation: _____

Department / institution: _____

Supervisor signature: _____ Date: _____

Head of department / authorized signatory: _____

Signature and seal: _____ Date: _____

Acknowledgement

This report recognizes the contributions of the maintainers of the open-source software and browser standards used by Ordeal, including React, TypeScript, Express, TensorFlow.js, Chart.js, and the web platform documentation cited in the references. The Ordeal repository and its contributors are the primary sources for the implementation described here. Personal acknowledgements to a supervisor, department, institution, or project team have not been added because those details were not supplied.

Abstract

Ordeal is a web-based examination and proctoring platform that brings candidate exam delivery and proctor review into a single full-stack application. The candidate workflow includes exam selection, browser camera and microphone permission requests, timed multiple-choice answering, incident logging, submission, and a report card. The proctor workspace provides candidate status views, analytics, CSV export, and incident-evidence review. The application is built with React and TypeScript, served by Express, styled with Tailwind CSS, and uses Chart.js for visualizations. TensorFlow.js COCO-SSD performs browser-side object detection when the model is available; additional monitoring outputs are generated by image-pixel, position, brightness, frame-difference, and audio-level heuristics.

The report documents the project's background, requirements, architecture, algorithms, implementation, and verification. Repository analysis confirms source paths for exam/session/violation APIs and client workflows; the production build and TypeScript check passed in the inspected environment after dependency installation with npm's legacy peer-dependency resolution. The report also records material engineering considerations: the login is a client-side role selection rather than server authentication, application records are held in process memory, captured session and incident data are sent to the API, and heuristic or fallback scores are not evidence of validated identity or liveness detection. No automated behavioral test suite or model-accuracy evaluation was present in the repository. Results are therefore reported only where directly verified, and no unmeasured performance figures are claimed.

Keywords: online examination, remote proctoring, web application, React, TypeScript, Express, browser-based monitoring, object detection, software engineering.

Table of Contents

Declaration
Certificate
Acknowledgement
Abstract
List of Figures
List of Tables
List of Abbreviations
Chapter 1. Introduction
- 1.1 Preliminaries
- 1.2 Problem statement
- 1.3 Motivation
- 1.4 Aim and objectives
- 1.5 Scope and constraints
- 1.6 Project contribution
- 1.7 Organization of the report

Chapter 2. Literature Survey and Background
- 2.1 Review scope
- 2.2 Online assessment and the proctoring lifecycle
- 2.3 Automated monitoring approaches
- 2.4 Assessment architecture and scoring integrity
- 2.5 Object detection and browser media APIs
- 2.6 Privacy, security, and fairness

2.7 Web application architecture
2.8 Comparative analysis
2.9 Related literature summary
2.10 Research gap and project position
Chapter 3. Proposed Methodology and System Design
3.1 Problem formulation and report method
3.2 Functional requirements
3.3 Non-functional requirements
3.4 Architecture
3.5 Data flow and principal entities
3.6 API design
3.7 Candidate examination workflow
3.8 Security and privacy considerations
3.9 Algorithms and monitoring logic
3.10 Evaluation method
Chapter 4. Implementation
4.1 Technology stack
4.2 Frontend modules
4.3 Server modules and storage
4.4 Detection and monitoring implementation
4.5 Exam authoring, randomization and reporting
4.6 Build and run instructions
4.7 Software and device requirements
4.8 Runtime configuration and data handling
4.9 Expected outcomes and acceptance criteria
Chapter 5. Results and Discussion
5.1 Implemented outcomes
5.2 Verification environment and results
5.3 Functional verification matrix
5.4 Performance and detection metrics
5.5 Requirements traceability
5.6 Limitations and risks
5.7 Viable solutions and system roadmap
5.8 Discussion
Chapter 6. Conclusion and Future Scope
6.1 Conclusion
6.2 Future scope
References
Appendix A. Source Code Organization
Appendix B. Candidate Examination Workflow
Appendix C. Reproduction Notes
Appendix D. Verification Checklist
Appendix E. Originality and Publication Materials

List of Figures

- Figure 3.1 Implemented high-level architecture
- Figure B.1 Candidate examination workflow

- Figure 6.1 Recommended target architecture (future work)

List of Tables

- Table 1.1 Project objectives and implementation evidence
- Table 2.1 Selected literature and technical guidance
- Table 3.1 Principal functional requirements
- Table 3.2 Principal non-functional requirements
- Table 3.3 Core REST API routes
- Table 3.4 Implemented algorithms and signal limitations
- Table 4.1 Technology stack
- Table 4.2 Software and device requirements
- Table 4.3 Expected outcomes and acceptance criteria
- Table 5.1 Build and static verification results
- Table 5.2 Functional verification matrix
- Table 5.3 Evaluation metrics and evidence status
- Table 5.4 Requirements traceability and verification status
- Table 5.5 Major implementation risks and limitations
- Table 5.6 Viable solutions and implications
- Table 6.1 Prioritized implementation roadmap

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AI	Artificial Intelligence
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
CV	Computer Vision
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MCQ	Multiple-Choice Question
OCR	Optical Character Recognition
REST	Representational State Transfer
SPA	Single-Page Application
TF.js	TensorFlow.js
UI	User Interface
Vite	Frontend development and build tool

Chapter 1. Introduction

1.1 Preliminaries

Online assessments allow candidates to answer questions through a browser and allow instructors to review results remotely. They also create operational challenges: candidates may need access to suitable devices and a

reliable connection; supervisors need an overview of activity; and the institution must decide how to manage identity, evidence, privacy, accessibility, and disputed incidents. A browser can provide access to a camera, microphone, visibility state, and fullscreen controls, but those capabilities depend on browser permissions, device support, and user-agent behavior. For example, getUserMedia requires user permission and a secure context in browsers that implement the API [3].

Ordeal brings together an examination interface, candidate monitoring widgets, a proctor dashboard, and incident review in a single web application. The repository describes a local-first vision, yet the implemented client also posts session and incident data, including image snapshots, to its Express server. This report therefore describes Ordeal as a browser-assisted, API-connected web application rather than as a strictly offline application.

1.1.1 Project overview

The project uses a React single-page interface served by an Express server. In development, the server mounts Vite middleware; in production it serves built static assets. The server exposes routes for exams, sessions, violations, question generation, document scanning, anti-spoof analysis, and diagnostics. The candidate and proctor are separate interface roles selected through the login screen. The repository README names the product an “Autonomous Neural Anti-Spoofing Examination Platform”; the implementation review in this report uses narrower terminology where source code does not support stronger claims [1].

1.1.2 Scope of this report

The report covers the repository snapshot at commit 86d1e38, its README, package metadata, frontend components, utility modules, and Express routes. It documents source-visible design and validation only. It does not claim real-world detection performance, institutional deployment, security certification, legal compliance, or user-study outcomes. The author name and roll number were supplied; degree, institution, supervisor, and department details were not supplied and are not invented.

1.2 Problem statement

A basic online quiz is straightforward to implement, but a proctoring workflow needs to combine test delivery, user feedback, incident handling, and result review. An integrated application can support these flows. However, labels such as “liveness verified,” “secure exam,” and “tamper-proof certificate” imply measurable guarantees. If those labels are produced by fixed thresholds, hard-coded fallbacks, or unsigned client state, the interface may communicate more certainty than the software provides.

Ordeal’s principal engineering challenge is therefore twofold. It must coordinate a candidate examination with a proctor review experience, and it must state clearly what its monitoring signals can and cannot establish. The source shows a functioning breadth of interface features but also identifies gaps in authentication, persistence, scoring consistency, and empirical validation.

1.3 Motivation

The project is motivated by the value of integrating a modern web stack with browser media APIs, lightweight computer-vision inference, exam workflows, charts, and administrative interfaces. It provides a concrete platform for studying client-server data flow and the distinction between machine-learning inference and manually designed heuristics. It also offers a practical case for examining how online assessment systems should communicate uncertainty, accessibility considerations, and privacy implications.

1.4 Aim and objectives

Aim: To provide a browser-based system that coordinates online examination delivery with candidate monitoring interfaces and a proctor review workflow, while keeping scoring, data handling, and monitoring claims grounded in verifiable implementation.

The repository-level objectives and current evidence are summarized below. “Implemented” describes the presence of a code path or interface, not demonstrated production reliability.

Table 1.1. Project objectives and implementation evidence

Objective	Evidence in repository	Status / qualification
Provide candidate and proctor experiences	Role-switched React UI and separate workspaces	Implemented; role selection is client-side, without server authentication
Deliver timed multiple-choice exams	Exam selection, question navigation, timer, submit flow	Implemented; server/client shuffle and scoring can diverge
Offer browser-based monitoring	Camera/mic permission flow, frame analysis and incident records	Implemented with COCO-SSD and custom heuristics; accuracy not evaluated
Give proctors review and analytics tools	Status cards, charts, CSV and evidence review	Implemented; some values are estimated or illustrative
Manage exams and questions	Express endpoints and question-builder/scanner interfaces	Partial; client and server IDs are not reconciled consistently
Protect privacy and preserve records	Browser-side processing plus REST API	Not complete; payloads go to server and data is memory-only
Validate software behavior	Build and TypeScript check scripts	Build/type-check verified; no automated behavioral tests

1.5 Scope and constraints

The report covers the public repository revision 86d1e38 reviewed on 6 November 2026: the README, package metadata, React components, TypeScript types, utility modules, Express routes, and build scripts. It describes source-visible product behavior and compile-time checks. It does not claim deployment at an institution, measured cheating reduction, validated biometric identification, or legal certification. Personal, institutional, and supervisory information is limited to the author name and roll number supplied for this report.

1.6 Project contribution

Ordeal integrates exam authoring and delivery, candidate monitoring interfaces, a proctor review workspace, analytics, and incident evidence handling in a single browser-based application. The project also demonstrates the distinction between pretrained object detection and application-specific heuristic signals, and it provides a concrete setting for studying data flow, client-server consistency, privacy disclosure, and software verification in remote assessment.

1.7 Organization of the report

Chapter 2 surveys online assessment, automated monitoring, browser media APIs, and published privacy research. Chapter 3 specifies requirements, system design, data flow, algorithms, and evaluation method. Chapter 4 documents the implementation and operating requirements. Chapter 5 reports the checks performed, evidence status, and risks. Chapter 6 concludes and proposes future enhancements. Appendices contain repository structure, workflows, reproduction instructions, and a verification checklist.

Chapter 2. Literature Survey and Background

2.1 Review scope

This chapter reviews the development context for online assessment and remote proctoring, the technical role of browser media and object detection, and published concerns about privacy, transparency, and fairness. It uses a small, focused literature set rather than claiming a systematic review. The purpose is to situate Ordeal's implementation choices and identify engineering requirements; none of the cited studies evaluates Ordeal itself.

2.2 Online assessment and the proctoring lifecycle

A remote assessment system typically coordinates exam authoring, candidate authentication, pre-exam readiness, exam delivery, monitoring, incident review, scoring, and result reporting. These responsibilities may be performed by a browser, a server, a human proctor, or a combination of them. Heinrich's review maps common proctoring phases from registration and pre-exam checks through monitoring, intervention, and post-exam review, while noting that public descriptions often leave data processing and AI decision roles unclear [6]. This lifecycle is a useful design checklist for Ordeal, but the presence of an interface for a phase is not the same as a verified service outcome.

For assessment correctness, the server should own canonical exam/question/option identities, attempt state, and grading rules. A randomized presentation should retain a stable mapping to the canonical options. The client in Ordeal shuffles options and submits indices, while the server grades against the original ordering. The workflow is implemented, but this mapping issue must be resolved before score reliability can be asserted.

2.3 Automated monitoring approaches

Remote proctoring systems can combine live human supervision, recorded sessions reviewed later, browser event monitoring, object/person detection, identity checks, audio/video signals, or restricted-browser features. Nigam et al.'s systematic review of 43 papers from 2015–2021 describes a range of AI and non-AI architectures and identifies privacy/security, ethical, trust, and technology-use concerns [8]. Heinrich's later review likewise emphasizes transparency in the use of AI and decision processes [6]. These findings support a conservative design principle: monitoring outputs should be treated as indicators for human review, not as automatic proof of misconduct.

Ordeal implements page visibility/focus listeners and a browser-side object detector when available, plus custom pixel/audio heuristics. These provide observable signals for an interface, but do not create a complete lockdown environment or establish that a candidate violated a rule. A proctor-facing review process is therefore more appropriate than a deterministic adverse outcome based solely on heuristic scores.

2.4 Assessment architecture and scoring integrity

A browser-based assessment typically separates the user interface from the authoritative exam and grading service. A server should own the canonical question bank, attempt identity, answer mapping, and final score. When answer order is randomized, the randomized presentation must be recorded or translated back to canonical question and option identities before submission. Otherwise, the same displayed answer index can point to a different choice in the server's original order.

Ordeal implements the candidate and proctor interfaces and a server-side submission route. However, the client shuffles options and submits indices, while the server grades against its original question ordering. This design

gap is documented in Chapter 5; the project's current result should not be treated as a reliable assessment score in randomized exams.

2.5 Object detection and browser media APIs

TensorFlow.js publishes browser-compatible models for common vision tasks. Its model directory describes COCO-SSD as object detection that localizes and identifies multiple objects in an image [2]. Ordeal uses the COCO-SSD package with a lightweight MobileNet v2 base. Such a detector can report classes represented by its model; it does not, by itself, verify candidate identity, determine gaze direction, prove liveness, or distinguish a physical face from a replay attack.

The repository supplements object detection with custom image statistics. The active utility samples pixels, estimates skin-like regions, uses approximate box position for gaze, compares frames, and derives a score from thresholds. These are heuristics, not validated biometric measurements. The reported score is a program-generated indicator; no calibration study, false-accept/false-reject analysis, or independent ground-truth evaluation is included in the repository.

2.6 Privacy, security, and fairness

Camera and microphone access is mediated by browser permission. The MediaDevices getUserMedia method prompts the user and can reject access when permission is denied or media devices are unavailable; it is restricted to secure contexts in supported browsers [3]. A website should explain why it requests access, what data is captured, where it is sent, how long it is retained, and how a user can stop capture.

The Page Visibility API signals when a document becomes visible or hidden, including common cases such as switching tabs or minimizing the window [4]. It is a useful event source for user-interface behavior, but it does not prevent a candidate from using another device or establish that a violation occurred. Ordeal's source includes focus and visibility listeners, while its interface presents stronger lockdown wording than the implemented controls justify.

Privacy and trust are system requirements because camera and microphone data are personal and context-rich. In a USENIX SOUPS study, Balash et al. analyzed reviews and surveyed 102 participants; participants expressed concern about the amount and personal nature of data shared with proctoring services, while also recognizing a trade-off in assessment integrity and remote learning [7]. The finding concerns participants in that study and should not be generalized to all students. For Ordeal, it motivates clear consent, purpose limitation, data minimization, documented retention, access controls, and candid disclosure of where images and session data are sent.

2.7 Web application architecture

React is used to compose interactive screens and component state, TypeScript describes data structures, Vite provides development and build support, and Express provides REST routes. Chart.js is used for score, incident, and topic visualizations; its documentation describes it as a JavaScript charting library with framework integration support [5]. These technologies are appropriate for an application that brings together forms, dashboards, and dynamic charts. They do not supply persistence, server authorization, or correctness of application-specific calculations automatically.

2.8 Comparative analysis

Approach	Strength	Limit / implication for Ordeal
Manual invigilation	Human context and judgment	Costly to scale; not the focus of this codebase
Browser event monitoring	Simple integration with visibility/focus events	Signals are incomplete and not proof of misconduct
Pretrained object detector	Can identify supported object classes in images	Cannot establish identity or robust liveness by itself
Custom pixel heuristics	Lightweight; easy to inspect and demo	Sensitive to camera, lighting, skin tone, scene and thresholds; not validated
Server-side storage	Can centralize records and grading	Current server arrays are process-local and reset on restart
Human-reviewed evidence	Supports review rather than automatic penalty alone	Requires reliable provenance, privacy rules and durable evidence handling

2.9 Related literature summary

Table 2.1. Selected literature and technical guidance

Source	Approach / sample	Relevant finding	Design implication for Ordeal
Nigam et al. (2021) [8]	Systematic review; 43 papers from 2015–2021	AI and non-AI system designs coexist; privacy, security, ethics, trust and training remain challenges.	Separate model capabilities from product claims; define safety and privacy requirements.
Balash et al. (2021) [7]	Browser-extension reviews and online survey (n=102)	Participants raised concerns about amount and sensitivity of proctoring data and described institutional trade-offs.	Use clear, specific consent and minimize captured data; do not assume universal acceptance.
Heinrich (2025) [6]	Review of publicly available descriptions of 33 systems	Transparency about data processing, AI roles, and decision-making is frequently limited; review and open standards matter.	Document data flow, decision roles, and evidence-review process.
TensorFlow.js COCO-SSD [2]	Pretrained browser object detector	Detects and localizes supported object classes in images; object detection is not person identity or liveness verification.	Describe Ordeal's model output narrowly and avoid biometric claims.
MDN browser API guidance [3] [4]	Platform documentation	Media access is permission/secure-context dependent; visibility events report page state.	Handle permission denials and treat browser events as partial signals.

2.10 Research gap and project position

The literature supports the need for integrated assessment workflows while highlighting the operational and ethical difficulty of reliable automated supervision. Ordeal's contribution is an application-level integration of candidate exam delivery, browser event/media handling, a proctor workspace, incident review, and reporting. The inspected repository does not provide a validated detector benchmark, a user study, or a production security assessment. Accordingly, this report presents the application's implemented features and identifies evaluation work still required; it does not infer system efficacy from the existence of components.

Chapter 3. Proposed Methodology and System Design

3.1 Problem formulation

The project can be framed as the design of a web application that allows a candidate to select and complete a multiple-choice examination, records selected activity and answers, and gives a proctor an interface for reviewing candidates and incident records. A second design goal is to experiment with browser-side object detection and image/audio heuristics while preserving enough separation between client presentation and server-managed exam state for the workflow to be extended.

The methodology for this report combines repository-based implementation analysis with a requirements-and-evidence review. The code paths were traced from candidate/proctor screens through API requests to server data structures; package metadata and scripts were checked; and build/type-check outcomes were recorded separately from behavioral and model-performance claims. Published sources are used for context, not as evidence of Ordeal's own effectiveness. No user study, training run, benchmark, deployment audit, or institutional evaluation was conducted for this report.

3.2 Functional requirements

Table 3.1. Principal functional requirements

ID	Requirement
FR-1	The application shall expose candidate and proctor interface modes.
FR-2	A candidate shall be able to select an exam and begin an attempt.
FR-3	A candidate shall answer, navigate, flag and submit multiple-choice questions.
FR-4	The examination interface shall display a timer and selected monitoring indicators.
FR-5	The client shall be able to submit session information, answers and incident records to the API.
FR-6	A proctor shall be able to inspect candidate status and review violation records.
FR-7	A proctor shall be able to view cohort charts and export selected data in CSV form.
FR-8	The application shall provide a post-submission report card with answer and topic summaries.
FR-9	The application shall support creating exams/questions through UI workflows and REST endpoints.

3.3 Non-functional requirements

Table 3.2. Principal non-functional requirements

Category	Requirement / evaluation
Usability	The browser UI should guide candidates through start, answering and submission.
Compatibility	A modern browser is needed; camera/microphone features require user permission and compatible APIs.
Performance	The app should render promptly and run monitoring without blocking the exam; repository has no benchmark results.

Category	Requirement / evaluation
Privacy	The product should clearly identify captured information and its transmission/retention. Current code posts snapshots and session data to the server.
Security	Authentication, authorization, input validation, secret management and secure durable storage are necessary for deployment; the current implementation does not provide the full controls needed for sensitive deployment.
Reliability	Exam attempt identity, question mapping and grading should remain consistent under randomization and errors.
Maintainability	Typed component and utility modules separate major screens and application data structures.
Auditability	Proctor decisions and evidence should be reviewable; current records are in-memory and non-durable.

3.4 Architecture

Figure 3.1 presents a simplified view of the code as implemented. Candidate and proctor screens run in the browser. The browser performs some local frame analysis but communicates with an Express API for exams, sessions, and violations. Module-level arrays hold the records in process memory. This architecture supports the integrated workflows, but does not provide database durability or meaningful multi-instance consistency.

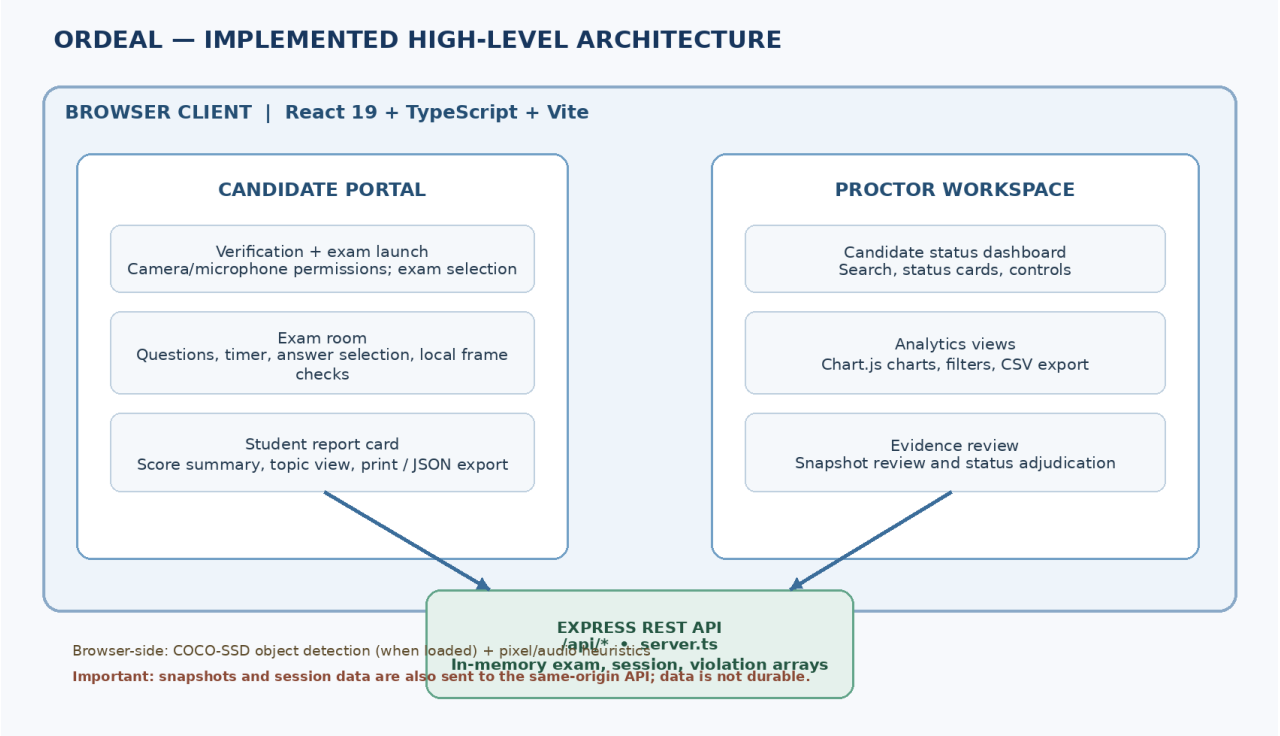


Figure 3.1. High-level architecture based on the inspected repository. The diagram distinguishes local browser processing from API-backed data exchange.

3.5 Data flow and principal entities

The application defines TypeScript structures for exams, questions, candidate sessions, violations, candidate users, and liveness metrics. The server seeds example exam, session, and violation records and keeps each category in module-level arrays. Candidate session startup associates a candidate with an exam; the candidate

then submits answer values and the server calculates a score. Incident logging can include a base64-encoded snapshot and bounding boxes. A proctor review action updates the violation status and notes in the in-memory object.

Because arrays are process-local, all changes disappear when the server process restarts. The application does not use a database or persistent file store in the inspected snapshot. The frontend also keeps user and session state in React component state rather than durable browser storage.

3.6 API design

The principal endpoints implemented in server.ts are summarized below. Routes are not protected by server-side authentication or authorization middleware in the inspected code. The route names differ from some entries in the README; this table reflects the actual server routes [1].

Table 3.3. Core REST API routes implemented in server.ts

Method	Route	Purpose / note
GET	/api/exams	List the in-memory exam collection.
GET	/api/exams/:id	Read one exam.
POST	/api/exams	Create an exam; server assigns its own identifier.
PATCH	/api/exams/:id/settings	Update settings for an exam.
POST	/api/exams/:id/questions	Append a question; server assigns its own ID.
POST	/api/sessions/start	Create a session.
POST	/api/sessions/submit	Submit answers and calculate score.
GET	/api/sessions	List sessions.
POST	/api/violations/log	Record an incident and optional snapshot.
GET	/api/violations	List incidents.
PATCH	/api/violations/:id/review	Update incident review status/notes.
POST	/api/generate-distractors	AI-backed endpoint when configured; otherwise fallback behavior.
POST	/api/scan-document	AI-backed document scan endpoint when configured.
POST	/api/verify-anti-spoof	AI-backed image analysis when configured; otherwise simulated fallback.
GET	/api/diagnostics	Return diagnostic information.

3.7 Candidate examination workflow

The current source flow begins with a candidate profile and exam selection. StudentVerification requests access to camera and microphone and offers a sequence of UI checks; a skip action is also available. The app calls the session-start endpoint and transitions into ExamRoom. The candidate answers questions, the client collects monitoring signals, and selected incidents are sent to the API. On submission, the server grades the submitted answer indices against its canonical exam and returns a score. The client then displays the report card. Appendix B, Figure B.1 summarizes the interaction at a high level.

3.8 Security and privacy considerations

A production system would require stronger user authentication, role-based authorization enforced on the server, durable and access-controlled storage, explicit evidence-retention policy, secure deployment configuration, and a documented appeal process. Webcam images and session information are sensitive: a UI

that says processing is local does not change the fact that snapshots are POSTed to the server. Where the optional Gemini endpoints are configured with GEMINI_API_KEY, image and document content may be sent to an external AI service by server-side routes. Those routes exist even though the principal scanner and verification UI flows do not call all of them in the current code.

3.9 Algorithms and monitoring logic

The following table summarizes the main algorithmic paths visible in the repository. It describes implemented logic rather than asserting clinical, biometric, or security validity. The browser detector depends on successful model loading and compatible execution support; the supplementary signals are threshold-based rules.

Table 3.4. Implemented algorithms and signal limitations

Algorithm / path	Implemented method	Output and engineering consideration
Question order shuffle	Fisher–Yates-style in-place swaps using Math.random().	Produces a randomized display order; stable option identity is not reconciled consistently at server grading.
Object detection	TensorFlow.js loads COCO-SSD with a MobileNet v2 base and evaluates camera frames when initialized.	Supported object classes and boxes only; not identity, liveness, or gaze verification.
Frame / pixel heuristics	Canvas samples pixels and compares luminance, frame changes, approximate regions and local contrast against fixed thresholds.	Heuristic counters/scores are environment-sensitive and have no reported calibration or accuracy study.
Audio signal	Web Audio analyser magnitude is reduced to a level indicator.	An audio-level value is not speech recognition, speaker identification, or proof of an unauthorized conversation.
Incident capture	Selected monitoring events are packaged with metadata and optional encoded image evidence and posted to the API.	Sensitive evidence is transmitted to the server; retention and access policy are not implemented in repository storage.
Fallback anti-spoof route	When the optional AI call is not available or fails, the server returns simulated positive values.	Fallback output is not derived from the submitted image and must not be interpreted as verification.

3.10 Evaluation method

Verification is divided into four evidence classes: (1) static source inspection establishes that a code path or route exists; (2) build/type checking establishes that the checked source compiles under the recorded dependency setup; (3) functional testing requires an executed user/API scenario with observed expected output; and (4) model or system evaluation requires test data, a defined ground truth, metrics, and repeatable conditions. This review verifies the first two classes, identifies but does not claim completion of the third, and finds no evidence for the fourth in the repository.

Chapter 4. Implementation

4.1 Technology stack

Ordeal is implemented as a browser-based full-stack application. The selected libraries support the visual interface, server routes, charting, and client-side computer vision; the business rules and persistence behavior remain application responsibilities.

Table 4.1. Technology stack

Layer	Technology	Role in project
Frontend	React 19, TypeScript 7	Single-page interface and typed application logic.
Styling	Tailwind CSS 4, CSS	Layout, theme tokens, responsive presentation.
Development/build	Vite 8, @vitejs/plugin-react	Development middleware and production asset build.
Backend	Node.js, Express 4, tsx	REST endpoints and combined development/production server.
Vision	TensorFlow.js 4, COCO-SSD 2	Browser-side object detection when the model is ready.
Charts	Chart.js 4	Analytics and topic visualizations.
Icons/animation	Lucide React, Motion	Interface iconography and interaction effects.
Optional AI service	Google GenAI client	Server routes for document scanning, distractor generation and anti-spoof analysis when configured.

4.2 Frontend modules

App.tsx coordinates the authenticated-looking UI state, data fetches, exam selection, callbacks, and role-dependent views. LoginPage provides candidate/proctor tabs and sample-account shortcuts. StudentVerification performs pre-exam setup and starts an attempt. ExamRoom displays questions, timer, local monitoring indicators and submission controls. StudentReportCard summarizes a submission. ProctorDashboard, ProctorAnalytics and ViolationEvidenceReview support administrative review. QuestionBuilderModal and DocumentScannerModal offer exam-authoring workflows.

The components are substantial, but some labels and actions are demonstrative. The login screen creates a client-side user object without verifying a password. Proctor dashboard cards use avatar images rather than remote live camera streams. Several action controls update temporary UI feedback only. Analytics include estimated values and demonstration data when source records are absent.

4.3 Server modules and storage

server.ts initializes Express JSON parsing, optional Google GenAI configuration, sample data arrays, REST routes, and the Vite or static-file server path. The server stores exam, session and violation data in memory. There is no database dependency or persistence layer in package.json. The API does not enforce authentication or role access. Consequently, the checked-in server implementation does not yet provide the persistence and access controls expected for deployment with sensitive assessment records.

4.4 Detection and monitoring implementation

LocalNeuralDetector initializes TensorFlow.js and loads the COCO-SSD lite MobileNet v2 model. When model inference is available, it can produce common-object detections, including supported person and object classes. Other outputs are derived by a custom canvas routine: sampled RGB values estimate skin-like pixels; approximate boxes feed position-based gaze categories; luminance changes increment blink-like counters; frame differences contribute to a static-image heuristic; local contrast contributes to a screen-pattern heuristic; and audio analyser magnitude is reduced to an average level. The utility then applies fixed penalties and thresholds to construct a score and verdict.

Those signals should not be described as a validated facial liveness model, eye tracking, speech recognition, or forensic anti-spoof verification. COCO-SSD is an object detector, not a face-identity system [2]. The optional server endpoint with a configured Gemini key can submit a captured image for AI analysis; when no key is present or analysis fails, the fallback returns a randomly generated high liveness score and fixed positive verdict. That fallback does not evaluate the image.

4.5 Exam authoring, randomization and reporting

The scanner accepts text, files, or webcam imagery and previews questions. The active local path parses text with regular expressions; image inputs may be replaced with fixed sample questions instead of OCR. Some document formats are read as plain text rather than decoded by a PDF/DOCX parser. The question builder generates distractors from keyword-based local string pools and generic templates; it does not verify semantic relevance or answer correctness.

Question shuffling uses Fisher–Yates swaps with Math.random(). This is a conventional algorithmic shuffle, but not seeded deterministic randomization. The server retains the original exam order, while scoring compares submitted option indices with those original indices. Consequently, enabling option randomization can produce score mismatches. The report card combines server scores with client-side question details, so headline and per-question results can disagree in some cases.

4.6 Build and run instructions

The package declares npm scripts for development (npm run dev), production build (npm run build), type checking (npm run lint), and a server start command (npm start). The README recommends a current Node.js installation and a browser with WebGL and camera/microphone support. Running the actual app requires dependency installation and permission to use device media. The repository contains an example environment file with GEMINI_API_KEY and APP_URL settings; keys should be supplied through an appropriate secret manager, not committed to source control.

In the validation environment, a standard npm install failed because the declared Vite 8 dependency accepts esbuild versions ^0.27 or ^0.28 while package.json declares esbuild ^0.25. Installing with --legacy-peer-deps completed, after which npm run build and npm run lint both passed. The production bundle emitted a large-chunk warning; no runtime performance benchmark was executed.

4.7 Software and device requirements

The repository does not publish a quantified minimum-hardware specification or performance benchmark. The following table separates explicit software prerequisites from practical device needs inferred from the implemented camera/microphone and browser-inference workflows; it should not be read as a tested compatibility guarantee.

Table 4.2. Software and device requirements

Requirement	Role / status	Verification note
Node.js and npm	Install project dependencies and run package scripts.	README/package metadata provide run instructions; the peer-dependency conflict is documented in Chapter 5.
Modern browser with JavaScript support	Run the React/Vite web application.	Exact browser/version matrix was not tested.
Camera and microphone	Enable pre-exam media permission and monitoring widgets.	User permission and device availability are required; denial/error pathways need manual verification.

Requirement	Role / status	Verification note
Secure context for camera/microphone	Browser media APIs require a secure context in supported browsers.	Localhost is generally treated as secure by browsers; deployment must use HTTPS.
WebGL / browser inference support	Support the TensorFlow.js model execution path.	The README mentions WebGL support; actual GPU/CPU performance was not benchmarked.
Desktop/laptop with usable display, webcam and microphone	Candidate workflow and media capture.	Practical requirement inferred from the app's UI; no minimum CPU, memory, resolution, or network figures were measured.
Internet / API reachability	Load frontend assets/model and communicate with Express/API or optional external AI service.	Offline behavior and bandwidth tolerance were not systematically tested.

4.8 Runtime configuration and data handling

The example environment file names GEMINI_API_KEY and APP_URL. The key is optional for the external AI routes; a missing key can trigger simulated fallback behavior in the anti-spoof route. Secrets must not be committed to source control. Because session and incident payloads are sent to the Express server, deployments should define TLS, access restrictions, evidence retention and deletion, logging controls, and user-facing data notices before enabling real candidates.

4.9 Expected outcomes and acceptance criteria

The following outcomes define a practical completion and evaluation target for an online examination platform. "Current evidence" reports what the inspected repository establishes today; it does not convert an intended requirement into a claimed result.

Table 4.3. Expected outcomes and acceptance criteria

Expected outcome	Suggested acceptance criterion	Current repository evidence
Correct exam delivery	Candidate can start, answer, flag, submit, and receive a consistent report for a known test exam.	Relevant UI and API paths exist; complete end-to-end test was not performed.
Reliable score calculation	Server grades stable option identifiers and agrees with the candidate-facing review for every test case.	Known risk: shuffled client option indices can differ from server canonical ordering.
Useful proctor review	Incident payloads are traceable to an attempt and review decisions persist and are auditable.	Review UI/API paths exist; server storage is in memory and no audit/persistence test exists.
Transparent monitoring	Candidate sees precise media/data notices; model output, heuristic flags, and unavailable states are distinguishable.	Some monitoring signals exist; comprehensive notice, retention, and fallback semantics need work.
Measured detection performance	Held-out labeled data supports precision/recall/F1 and false-positive analysis across relevant conditions.	No labeled dataset, benchmark, or observed detector metrics found.
Reproducible delivery	Documented clean install and production build succeed without dependency conflicts.	Build/type check succeeded with --legacy-peer-deps; default dependency resolution reported ERESOLVE.

Chapter 5. Results and Discussion

5.1 Implemented outcomes

The inspected codebase implements an end-to-end application structure spanning candidate and proctor views, exam selection and authoring, timed question answering, session/incident APIs, charts, report cards, CSV export, evidence review, and browser media/detection signals. These are concrete software deliverables. Their presence is distinct from production deployment, security certification, or validated proctoring accuracy; those outcomes require separate evidence and are not claimed here.

The architecture diagram, route inventory, feature modules, and source tree provide evidence that the application is a substantial full-stack project rather than a single-screen mock-up. The most important unresolved engineering outcomes are authoritative exam/option mapping, server-side identity/access control, durable records, explicit media-data governance, and behavioral validation.

5.2 Verification environment and results

Table 5.1. Build and static verification results

Check	Method	Observed result
Source inspection	Review of README, source files, types and routes	Completed; implementation findings recorded.
Dependency install	npm install with default peer resolution	Failed with ERESOLVE due Vite/esbuild peer mismatch.
Dependency install workaround	npm install --legacy-peer-deps	Completed in inspected environment.
Production build	npm run build	Passed; emitted a large-chunk warning (about 1.68 MB uncompressed JS asset).
TypeScript check	npm run lint (tsc --noEmit)	Passed.
Automated behavioral tests	Search for test/spec files and package test scripts	No automated behavioral test suite found in the inspected repository.
Detection accuracy	No labeled data, benchmark, or evaluation harness in repository	Not measured; no accuracy claim can be made.

5.3 Functional verification matrix

The following table separates checks performed from recommended manual verification. No unexecuted case is presented as a pass.

Table 5.2. Functional verification matrix

Case	Expected behavior	Evidence status
Build and type-check	Assets build and TypeScript reports no errors	Passed in the inspected environment using the dependency workaround.
Candidate login validation	Invalid credentials cannot establish an authenticated role	Not provided: role selection is client-side and no server login was found.
Camera permission denied	Candidate receives a clear recovery path	Not manually tested; source requests browser media permission.
Exam shuffle and scoring	Displayed option maps to the canonical correct answer	Known source-level inconsistency; requires correction and automated tests.
Submission and report consistency	Server score and itemized report agree	Not fully tested; client/server paths can disagree.
Snapshot privacy	Candidate is informed of data transmission, access and retention	Design/documentation work required; source posts snapshot data to the API.

Case	Expected behavior	Evidence status
Restart recovery	Saved records survive server restart	Not supported by in-memory collections.
Evidence review	Review status and notes persist and surface consistently	Route/UI exist; behavioral and error-path tests absent.
Document scanning	Supplied source is accurately parsed into questions	Image/OCR path is not established; fallback/sample question behavior exists.

5.4 Performance and detection metrics

No runtime benchmark, labeled evaluation set, or instrumented performance study is included in the repository. Therefore, latency, throughput, uptime, detector precision, recall, false-positive rate, and fairness measures cannot be reported as observed results. For a future evaluation, ground-truth events should be annotated independently and the test set should cover lighting, camera position, device capability, background variation, and accessibility conditions.

Table 5.3. Evaluation metrics and evidence status

Metric	Definition / use	Current evidence
Build success	Whether the production build completes.	Observed: pass after dependency installation with --legacy-peer-deps.
Type-check	Whether TypeScript reports static errors.	Observed: npm run lint passed.
Precision	TP / (TP + FP): share of flagged cases that are true events.	Not measured; no labeled event set.
Recall	TP / (TP + FN): share of true events detected.	Not measured; no labeled event set.
F1 score	2 × precision × recall / (precision + recall).	Not measured; do not infer from heuristic scores.
False-positive rate	FP / (FP + TN), reported by relevant subgroup and environment.	Not measured; fairness evaluation absent.
Inference latency	Time from frame submission to displayed inference result.	Not measured; no benchmark harness.
Persistence recovery	Proportion of saved records retained after restart/failure.	Current storage is in memory; recovery is not supported.

For a binary monitoring decision, TP denotes a correctly detected event, FP a false alert, FN a missed event, and TN a correctly unflagged instance. Any threshold-tuned detector should be evaluated on held-out data and with confidence intervals; an overall metric alone can conceal subgroup or environmental disparities.

5.5 Requirements traceability

Table 5.4. Requirements traceability and verification status

Requirement group	Implemented evidence	Verification status
Candidate exam experience	Exam list, session start, timed question UI, submit flow and report card.	Source paths present; end-to-end behavior not covered by automated tests.
Proctor workflow	Dashboard, analytics, CSV export and violation review components.	UI/source present; live-camera streaming and durable review not established.
Monitoring	Browser media requests, COCO-SSD and custom heuristic signals.	Implementation present; detection performance not measured.
Exam/question administration	Builder and scanning interfaces, plus exam/question API routes.	Partially verified by source; client/server identifiers require reconciliation.
Privacy and security	Browser permission flow and optional environment-based AI integration.	Insufficient for sensitive deployment; API access control and durable privacy policy absent.

Requirement group	Implemented evidence	Verification status
Maintainability and delivery	Typed React/TypeScript modules and build/type-check scripts.	Build and type check passed; automated regression suite absent.

5.6 Limitations and risks

Table 5.5. Major implementation risks and limitations

Area	Current limitation	Impact
Authentication	Login constructs client-side profile data; no server login, password validation or tokens.	Any user can potentially reach role-specific API operations; not production-ready.
Authorization	API routes have no access-control middleware.	Candidate/proctor boundaries are cosmetic at the API layer.
Persistence	Module-level arrays reset on process restart.	No durable exam, attempt or evidence records.
Privacy	Session metadata and base64 snapshots are sent to Express; optional AI routes can send data externally.	Requires consent, minimization, retention controls and accurate disclosures.
Detection	One pretrained object detector plus custom thresholds/heuristics; no validated liveness/identity model.	Signals can be wrong; cannot justify automatic adverse decisions.
Fallback logic	Anti-spoof endpoint without AI key returns synthetic positive output.	A passing verdict is not evidence that the frame was inspected.
Question grading	Client shuffles answer options; server grades original indices.	Scores may be incorrect when randomization is enabled.
Client/server IDs	Exam/question creation can retain different local IDs than server-assigned IDs.	Session linkage and grading can target different records.
Document ingestion	Local path is regex parsing and canned fallback, not OCR of images/PDF/DOCX.	Imported content may not match user-supplied material.
Analytics	Some estimates, fixed baselines, and synthetic chart values are present.	Charts should not be interpreted as audited cohort measurements.
Testing	No automated test suite or performance/security evaluation.	Regression risk and operational behavior remain unknown.
Accessibility/fairness	No reported accessibility or demographic performance evaluation.	Camera, lighting, disability and environment differences may affect experience and heuristic signals.

5.7 Viable solutions and system roadmap

The following remediation plan translates the identified limitations into feasible engineering work. These are recommendations for future implementation; they are not represented as completed repository features. The implications describe expected engineering outcomes if the work is implemented and verified.

Table 5.6. Viable solutions, priority, and expected implications

Priority	Limitation / implication	Viable solution	Acceptance evidence
P0 — correctness	Randomized option indices can produce incorrect scores.	Assign immutable questionId/optionId values; shuffle only display order; submit option IDs and grade on the server. Record the presented order for audit.	Unit tests cover each option order; server score matches an independent oracle.
P0 — access control	Client-side role choice does not establish identity or permission.	Add server-side authentication, secure session/token handling, role-based middleware, rate limits, schema validation, and authorization tests on every sensitive route.	Unauthenticated and cross-role requests are rejected; negative tests cover all routes.

Priority	Limitation / implication	Viable solution	Acceptance evidence
P0 — durable records	In-memory arrays are lost at restart and cannot support reliable audit history.	Introduce a versioned relational persistence layer for users, exams, questions, attempts, answers, and incidents; add transactional writes, migrations, backups, and recovery tests.	Records survive restart; migration/backup/restore tests pass.
P0 — sensitive media	Snapshots/session payloads are sent to the server without a repository-defined retention policy.	Provide purpose-specific consent, data minimization, TLS, role-restricted access, encrypted evidence storage, retention/deletion controls, and clear notices for any external AI transfer.	Privacy review, network-flow audit, access tests, and deletion evidence are documented.
P1 — misleading fallback	Anti-spoof fallback may return synthetic positive scores when no image was evaluated.	Return explicit states such as unavailable/needs-review; never fabricate a passing score. Make human review the decision point and retain evidence provenance.	Failure-mode tests show unavailable status; no positive verdict is returned without analysis.
P1 — unmeasured monitoring	Heuristic outputs have no measured accuracy or fairness evidence.	Create a consented, labeled evaluation set; pre-register event labels/thresholds; report precision, recall, F1, false-positive rate, confidence intervals, and subgroup/environment analyses.	Reproducible evaluation report and independent review; no high-stakes action before validation.
P1 — content ingestion	Regex parsing and canned fallback do not establish faithful PDF/DOCX/image extraction.	Use format-aware document parsers and optional OCR; show extracted text and answers for human confirmation before storing questions.	Golden-file tests measure extraction fidelity; user confirms each imported question.
P2 — software assurance	Build/type check do not test API, scoring, permissions, or recovery behavior.	Add unit, integration, API-contract, browser end-to-end, accessibility, and security tests; run them in CI against locked dependencies.	CI verifies install/build/test; coverage includes critical paths and failure cases.
P2 — reproducibility and performance	Default dependency resolution fails and bundle size warning remains.	Align the declared esbuild range with the selected Vite peer requirements; maintain a committed lockfile; measure startup, route latency, and model inference; split/lazy-load large chunks.	Clean install/build succeeds without peer override; documented browser/device performance matrix.
P2 — equitable access	Camera/light/environment variance and accessibility are not evaluated.	Provide a non-camera accommodation path; test keyboard/screen-reader use, low bandwidth, lighting diversity, device variation, and disability-related needs with stakeholders.	Accessibility review and fairness analysis completed; alternate exam path documented.

5.8 Discussion

The project integrates candidate exam delivery, camera-assisted monitoring, a proctor workspace, analytics, and incident review. The application also compiles in the tested environment. These results establish substantial full-stack implementation; they do not establish that monitoring reliably detects misconduct or spoofing, that user identities are protected, or that grades are always correct.

The strongest next step is to improve correctness and trust boundaries before adding more visual features. Canonical exam and answer identifiers should be controlled by the server; shuffled option identities should be preserved and mapped during grading; all routes should enforce authentication and authorization; and persistence should be added with durable storage. In parallel, the monitoring pipeline should be evaluated using a documented dataset and error analysis, and its UI should avoid describing heuristic scores as biometric certainty.

Chapter 6. Conclusion and Future Scope

6.1 Conclusion

Ordeal is a full-stack web application for online examination and proctor review. Its checked-in implementation integrates React and TypeScript screens, an Express REST API, browser media APIs, TensorFlow.js object detection, custom monitoring heuristics, Chart.js analytics, report cards, question-authoring workflows, and incident review. The build and TypeScript check both passed in the inspected environment after dependency installation with npm's legacy peer-dependency resolution. The source provides concrete evidence of a substantial end-to-end project, including candidate, proctor, and server-side components.

The report also establishes the boundaries of what can be claimed. Current source does not provide server-side authentication/authorization or durable storage; randomized option mapping can diverge at scoring; document scanning includes non-OCR fallback paths; and detector/anti-spoof performance has not been validated on labeled data. These limitations do not erase the implemented application, but they matter for any deployment involving grades, identity, or sensitive media. The appropriate conclusion is that Ordeal is an implemented assessment platform whose reliability, security, and monitoring quality require further verification and engineering before high-stakes use.

6.2 Future enhancements

1. Create server-side authentication and enforce role-based authorization on every sensitive endpoint. Store passwords only as appropriate one-way hashes if account credentials are introduced.
2. Introduce durable database storage with schema validation, access controls, backup, retention and audit policies for exams, attempts, answers and incident records.
3. Make the server authoritative for exam, question and option identifiers. Persist the randomized order or map stable option IDs back to canonical answers before grading.
4. Fix client/server identifier reconciliation for scanner-created exams and authored questions; return and use server-canonical records.
5. Replace simulated anti-spoof fallbacks with explicit unavailable/error states. Avoid high-confidence positive verdicts when the image was not analyzed.
6. Validate object detection and any candidate monitoring measures using documented test data, metrics, environmental variation, accessibility review and independent human review. Do not automate disqualification from unverified heuristics.
7. Implement genuine OCR/document parsing with clear source previews and verify extracted questions and answers before saving them.
8. Add automated unit, integration and end-to-end tests for session creation, randomization, scoring, permissions, API errors, evidence review and persistence recovery.
9. Add privacy notices and consent, minimize captured content, restrict access, encrypt transport/storage, define retention and deletion, and disclose whether external AI services receive any material.
10. Improve bundle splitting and performance measurement; add responsive and accessible interaction checks for candidate and proctor workflows.

A practical implementation sequence is to first correct scoring and identifier consistency, then add authentication, authorization, durable storage, and auditable privacy controls. Next, build automated functional tests and measure performance. Finally, evaluate monitoring outputs with independent annotations and

accessibility/fairness review before presenting any score as decision-support evidence. This sequence prioritizes correctness and user trust over adding additional monitoring labels.

Table 6.1. Prioritized implementation roadmap

Phase	Work package	Deliverables	Exit criteria
P0 — correctness	Canonical exam data and grading	Stable question/option IDs, server-side score mapping, contract tests.	Known-answer end-to-end cases pass under multiple shuffle orders.
P0 — trust boundary	Identity and persistence	Server authentication/RBAC, validated requests, durable database, audit events, backups.	Negative access tests pass; restart/restore checks confirm record integrity.
P0 — privacy	Media governance	Consent and notices, minimization, encryption, evidence access/retention/deletion controls.	Documented data-flow and privacy review; deletion and access tests pass.
P1 — functional reliability	Question ingestion and service behavior	Format-aware parsing/OCR, user confirmation, robust API errors, stable client/server IDs.	Golden-file extraction and API integration tests pass.
P1 — software quality	Automation and performance	Locked dependency versions, CI pipeline, browser E2E suite, bundle and latency measures.	Clean install/build/test succeeds; published performance envelope.
P2 — responsible monitoring	Evidence-based model evaluation	Labeled test set, error/fairness analysis, human review, explicit unavailable states.	Independent evaluation reviewed; no adverse action from unvalidated signal.
P2 — adoption readiness	Accessibility and rollout	Keyboard/screen reader checks, non-camera accommodation, documentation and staged pilot plan.	Accessibility review and operational acceptance sign-off completed.

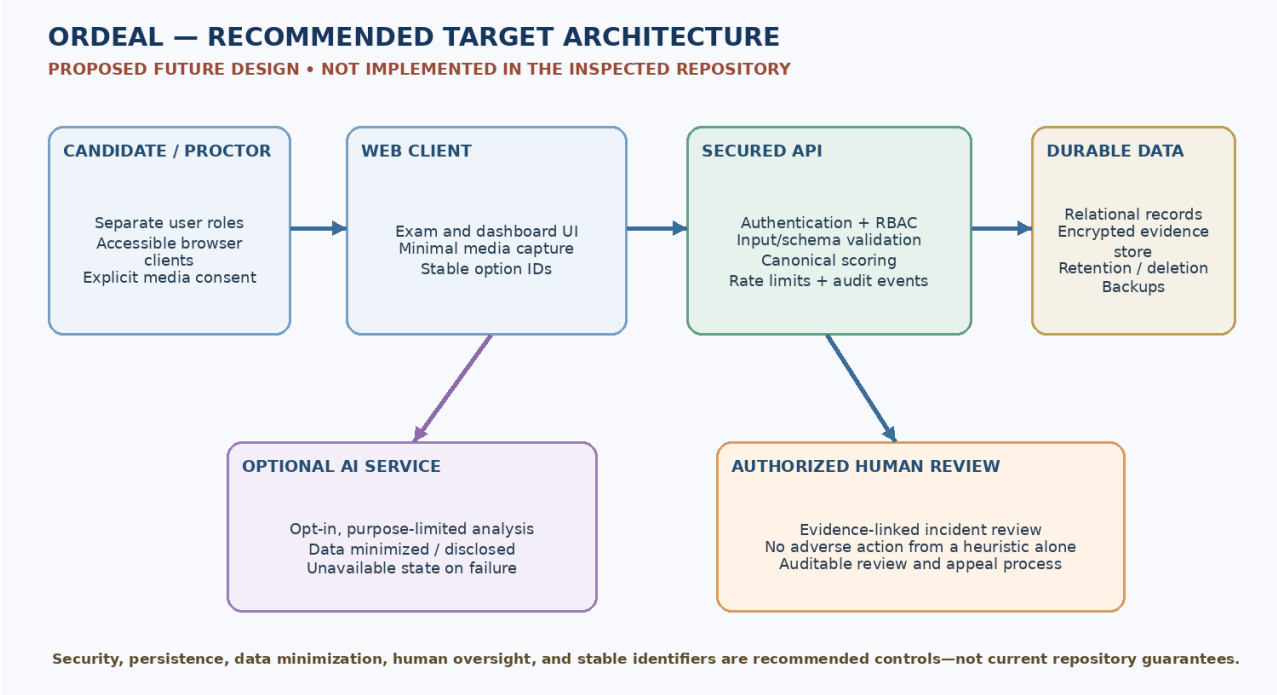


Figure 6.1. Recommended future architecture; the security, persistence, and human-review components shown are not implemented in the inspected repository.

References

[1] abhijeet0207hub. Ordeal: public source-code repository and README. Revision 86d1e38, reviewed 6 November 2026.. https://github.com/abhijeet0207hub/Ordeal

[2] TensorFlow.js. Models: COCO-SSD object detection documentation..
https://www.tensorflow.org/js/models

[3] MDN Web Docs. MediaDevices: getUserMedia() method..
https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[4] MDN Web Docs. Page Visibility API..
https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API

[5] Chart.js. Official documentation.. https://www.chartjs.org/docs/

[6] Heinrich, E. (2025). A Systematic-Narrative Review of Online Proctoring Systems and a Case for Open Standards. Open Praxis, 17(3), 485–499.. https://doi.org/10.55982/openpraxis.17.3.836

[7] Balash, D. G., Kim, D., Shaibekova, D., Fainchtein, R. A., Sherr, M., & Aviv, A. J. (2021). Examining the Examiners: Students' Privacy and Security Perceptions of Online Proctoring Services. Proceedings of SOUPS 2021, 633–652..
https://www.usenix.org/conference/soups2021/presentation/balash

[8] Nigam, A., Pasricha, R., Singh, T., & Churi, P. (2021). A Systematic Review on AI-based Proctoring Systems: Past, Present and Future. Education and Information Technologies, 26, 6421–6445..
https://doi.org/10.1007/s10639-021-10597-x

Appendix A. Source Code Organization

Path	Responsibility
server.ts	Express API, seeded in-memory state, optional AI client and dev/production server startup.
src/App.tsx	Top-level app state, API fetches, route/view switching and callbacks.
src/types.ts	TypeScript structures for exams, sessions, violations and monitoring data.
src/components/StudentVerification.tsx	Candidate setup, permission request, challenge UI and session startup.
src/components/ExamRoom.tsx	Timed exam, answer state, monitoring display, incident posting and submission.
src/components/ProctorDashboard.tsx	Candidate cards and proctor workspace navigation.
src/components/ProctorAnalytics.tsx	Chart.js analytics, filters, tables and CSV export.
src/components/ViolationEvidenceReview.tsx	Incident inspection and adjudication UI.
src/components/StudentReportCard.tsx	Candidate score and topic report.
src/utils/localNeuralDetector.ts	COCO-SSD integration and local pixel/audio heuristics.
src/utils/antiSpoofDetector.ts	Separate canvas-based heuristic detector.
src/utils/localDocumentScanner.ts	Text regex parsing and fallback question generation.
src/utils/localDistractorGenerator.ts	Keyword-based distractor strings.
src/utils/shuffle.ts	Client-side Fisher–Yates shuffle and answer-index remapping.

Appendix B. Candidate Examination Workflow

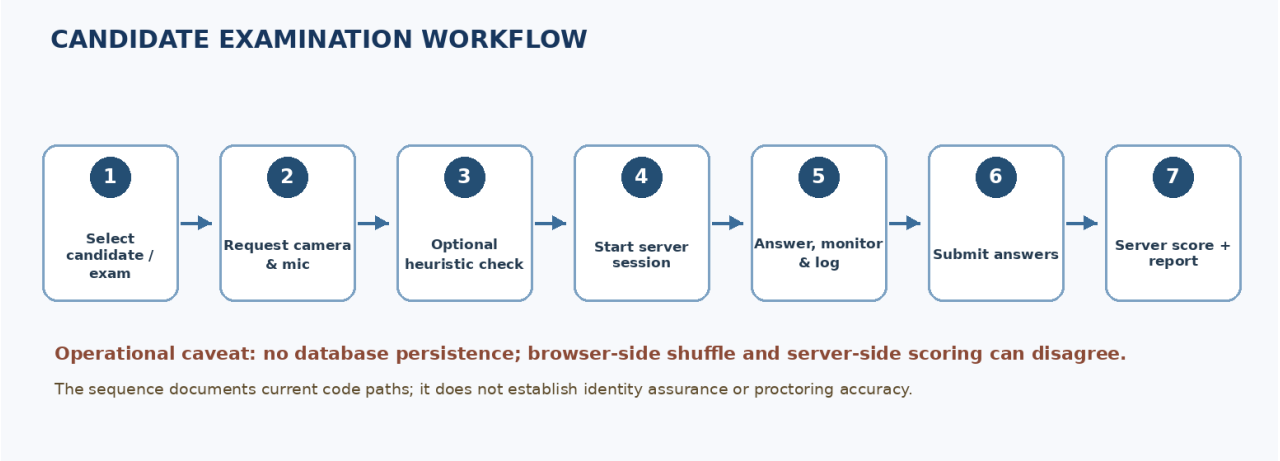


Figure B.1. Source-level candidate workflow; this sequence does not imply identity verification or validated monitoring.

Appendix C. Reproduction Notes

Repository: https://github.com/abhijeet0207hub/Ordeal. At the time of inspection, the working tree revision was 86d1e38. The package scripts include npm run dev, npm run build, npm run lint, and npm start. In the inspected environment, the normal npm dependency resolver stopped with ERESOLVE for the Vite/esbuild peer ranges; npm install --legacy-peer-deps succeeded. Then npm run build and npm run lint completed successfully. The build printed a warning about a large JavaScript bundle. This validation records compile-time checks only; it is not a security assessment, browser compatibility matrix, functional test report, or performance benchmark.

Appendix D. Verification Checklist

The following checklist is intended for a future controlled test session. Items are recommendations, not test results. Use a non-sensitive test account and synthetic exam data until authentication, authorization, and retention controls are defined.

Area	Recommended test	Evidence to record
Exam setup	Create an exam and questions, reload the page, and compare client/server identifiers.	Request/response payloads, persisted state, and expected canonical IDs.
Randomization and grading	Shuffle questions/options across repeated runs and submit known answer IDs.	Seed/order mapping and independent score calculation.
Media permissions	Test camera/microphone allowed, denied, unavailable, and interrupted.	Browser/version, user guidance, recovery path, and console/network errors.
API access control	Call every sensitive route with no identity, candidate identity, and proctor identity.	Expected HTTP status and role permission matrix.
Privacy and deletion	Record exactly which payloads leave the browser and how deletion is handled.	Network trace, notice text, retention schedule, deletion evidence.
Monitoring evaluation	Compare outputs to independently annotated clips across conditions.	Confusion matrix, precision/recall/F1, false-positive rate, confidence intervals.
Accessibility	Review keyboard, screen-reader, timing, lighting and alternative exam arrangements.	Test protocol, findings, and accommodations.

Appendix E. Originality and Publication Materials

No institutional similarity/plagiarism report, supervisor certificate, or project research paper/patent was supplied with the materials reviewed. This report does not claim a similarity percentage, publication, patent, or institutional approval. If required for submission, the author should obtain the institution’s approved similarity report and attach it as a separate verified document; any research paper or patent details should be added only after confirmation of authorship and status.

Academic fields supplied for this draft: Abhijeet Singh, Roll No. 2501920100016. Degree, institution, department, supervisor, head of department, and academic year were not provided and are intentionally not asserted.